Backtesting Tool — 아키텍처 & 운영 설계

강환국 퀀트 전략 자동매매 시스템 · 영속화 · 장애대응 · 이벤트 아키텍처 · 컨테이너 배포
생성일: 2026-04-23 · 버전 2.0 · Kafka/Redis/Docker/K8s 포함

목차

1. 시스템 개요
2. 영속화 — 메모리 상태를 DB로 이관
3. 자동화 테스트 — 전략 로직 검증
4. 장애 대응 — 기동 복구 · 중복 방지 · WS 백업
5. 성능 개선 — Redis 캐시
6. 이벤트 아키텍처 — Kafka
7. 컨테이너화 — Docker & docker-compose
8. 쿠버네티스 배포
9. 면접 예상 질의 & 답변

1. 시스템 개요

```
┌──────────────
```

2. 영속화 — 메모리 상태를 DB로 이관

Problem — 기존 구현은 OCO 포지션·전략 설정·실행 이력·감사로그를 모두 `ConcurrentHashMap` / `ArrayList` 메모리에만 저장. 서버 재시작 시 **전부 증발**. 운영 중 JVM 크래시, 배포 재시작, K8s pod eviction 시 진행 중인 OCO 감시가 손실되어 손절/익절이 실행되지 않는 치명적 리스크.

Solution

- H2 파일 모드 + JPA — 외부 의존성 최소화로 도입 장벽 낮춤 (인프라 추가 없이 영속화)
- 4개 엔티티: `OcoPositionEntity`, `StrategyConfigEntity`, `StrategyExecutionEntity`, `AuditLogRowEntity`
- 중첩 구조(TP legs, QuantSignal, Map<String,BigDecimal>)는 **JSON 컬럼**으로 저장 (타입별 AttributeConverter)
- 도메인 모델(`OcoPosition` 등)은 순수 POJO 유지 + `fromDomain()/toDomain()` 매퍼 — JPA 수명주기와 API 직렬화 분리

저장 시점

이벤트	영속화 동작
OCO 등록	PENDING_* 상태 → `repo.save()`
본주문 체결 확인	ACTIVE 전환 + stopLossPrice·TP trigger 계산 후 저장
손절/익절 실행	CLOSED 또는 PARTIALLY_CLOSED 저장 + closeReason 기록
전략 활성화/비활성화	`StrategyConfigEntity` upsert
전략 리밸런싱 실행	`StrategyExecutionEntity` insert (시계열)
API 호출	AuditInterceptor → `AuditLogService.record()` → DB insert

기동 복구

`@PostConstruct`에서 DB 로드:

```
OcoOrderService.recoverFromDb():
  1. repo.findByStatusIn(PENDING_ENTRY, PENDING_FILL, ACTIVE, PARTIALLY_CLOSED)
  2. KIS 잔고 조회 → 각 포지션 수량 대조
  3. 보유 수량 부족 = 외부 매도 감지 → CLOSED/EXTERNAL_CLOSE_DETECTED
  4. 나머지 → 메모리 Map + WS 재구독

QuantExecutionService.restoreFromDb():
  1. configRepo.findAll() → 모든 전략 상태 복원
  2. enabled=true면 스케줄러가 즉시 주기 체크 시작
  3. 실행 이력은 snapshot() 호출 시 lazy 로드 (최근 20건)
```

Trade-off

- H2는 단일 JVM 전제 (`AUTO_SERVER=TRUE` 로 완화). 수평 확장 시 Postgres 전환 필요.
- JSON 컬럼은 스키마 변경 유연하나 필드별 쿼리 불가. 대용량 검색 필요 시 별도 정규화.
- 감사로그 폭증 방지: 매일 03:00 KST 60일 경과 레코드 purge 스케줄러.

3. 자동화 테스트 — 전략 로직 검증

Problem — 전략 구현 10개의 정확성을 수동 브라우저 테스트로만 검증. 모멘텀 스코어 공식 오류, 경계 조건(월말·할로윈 4월/10월), 순위 합산 버그 등을 프로덕션 배포 후에야 발견 가능. 회귀 방지 불가.

Solution

- **JUnit 5 + Mockito** 기반 단위 테스트 38개
- 합성 가격 데이터 생성기 (`PriceSeriesBuilder`) — flat/linear 시나리오
- 팩터 전략은 `Mockito.mock(FundamentalDataService)` 로 재무 데이터 스텁
- 모든 전략 × 대표 시나리오 2~3개씩

테스트 커버리지

카테고리	테스트 수	검증 내용
QuantIndicators (지표 계산)	11	closestPriceAtOrBefore · momentumReturn · vaaScore · monthlySma · 룩백 부족 null 반환
Halloween 전략	7	월별 winter/summer 판정 · 4/10월 경계 · 리밸런싱 월 제한
DualMomentum	3	상대+절대 모멘텀 · risk-off 전환 · 월말 리밸런싱
VAA	3	all-positive → offensive · one-negative → defensive · topN=2 균등 분할
DAA	3	카나리아 0/1/2개 음수 시나리오별 0%/50%/100% offensive
LAA	3	정적 75% 고정 · SPY 10M SMA 판정 → QQQ vs 단기채
MagicFormula (팩터 대표)	3	ROA+PER 합산 랭킹 · 적자 기업 제외 · 분기 리밸런싱
FundamentalData	4	ROE/ROA/GPA/PSR 파생 · F-Score 계산 · null-safe
합계	**38개**	**전부 통과**

실행: `mvn test` · 평균 소요 ~6초

Trade-off

- KIS/DART 연동은 통합 테스트 미포함 (외부 API 과금·rate limit 문제) — Mockito로 대체.
- E2E 테스트는 스테이징 환경에서 수동 smoke test로 대체.

4. 장애 대응

4.1 기동 시 복구 (State Recovery)

앞서 2절에서 다뤘듯 DB에서 로드 + KIS 잔고 대조 + WS 재구독. 핵심은 "서버가 죽어있던 동안 외부 세계가 변했을 수 있다"는 가정으로 reconcile.

4.2 주문 중복 방지 (Idempotency)

Problem

- 스케줄러 중복 발화 (cron 재시작, K8s pod 재배치 시)
- 주문 API 네트워크 타임아웃 → 실제 접수됐는지 불확실
- 재시도 시 double-buy/double-sell 위험

3가지 방어 레이어

Layer 1 — 날짜 기반 중복 체크

```
if (kind == SCHEDULED && state.lastRebalanceDate.equals(today)) {
    log.info("Already rebalanced today, skipping");
    return null;   // 같은 날 재발화 차단
}
```

Layer 2 — KIS 미체결 조회 선행 (pre-flight dedup)

```
List<Map> open = kisTrading.getOpenOrders();
for (각 주문) {
    if (symbol + side + quantity 매칭) {
        return "중복 주문 방지 — 이미 KIS에 동일 조건 미체결 주문 존재";
    }
}
// 없으면 신규 주문 발송
```

Layer 3 — 네트워크 오류 후 reconcile

```
try {
    result = kisTrading.placeOrder(req);   // 타임아웃 발생
} catch (Exception e) {
    result = reconcileUnknownOrder(pos, e.getMessage());
    // 미체결 목록에서 symbol+qty 매칭 주문 찾아 성공 처리
    // 없으면 실패로 판정
}
```

Layer 4 — Redis 분산 락 (다중 인스턴스 대비)

```
distLock.runExclusive("quant:" + type, Duration.ofSeconds(30), () -> {
    runStrategy(type, SCHEDULED, true);
});
```

현 배포는 single-instance라 ConcurrentHashMap 동기화로 충분하나, 수평 확장 시 Redis 락으로 전환 가능한 인프라 구축.

4.3 WebSocket 끊김 대비

KIS WS 연결 끊김 → 실시간 틱 미수신 → **손절가 도달해도 감지 못 함**. 규칙상 자동 매도가 일어나야 하는데 그대로 방치되면 손실 폭 확대.

2중 방어

1. `KisWebSocketClient` 의 지수 백오프 재연결 + 기존 구독 자동 복원
2. `OcoOrderService.pollActivePositions()` — 60초 주기 폴백 폴링
   - ACTIVE/PARTIALLY_CLOSED/PENDING_ENTRY 포지션의 종목 현재가 `getQuote()` 조회
   - `onTick()` 재생 → 손절/익절 조건 평가
   - WS가 정상이어도 중복 트리거 안 됨 (remainingQuantity=0 체크)

4.4 기타 보호 장치

항목	처리
이메일 장애	`safeEmail()` try-catch 격리 — 주문 흐름 영향 없음
DB 장애	`persist()` 예외 로깅만 — 메모리 상태는 유지하여 기능 동작 계속
Kafka 장애	`EventPublisher` 는 fire-and-forget — 주문에 영향 없음
DailyLossGuard	당일 실현손실 한도 초과 시 신규 매수 전부 차단. 매도는 계속 허용
Graceful shutdown	`@PreDestroy` 로 WS 해지 · DB commit · KafkaTemplate flush
Health check	`/actuator/health` 에 KIS/DART/DB 상태 노출 — K8s probe 연동

5. 성능 개선 — Redis 캐시

Problem — KIS API는 초당 호출 한도 (실전 18 rps)가 있어 유저가 많아지거나 전략 10개를 동시 리밸런싱하면 **429 rate limit** 쉽게 도달. 동일 심볼 시세·잔고를 여러 지점에서 중복 호출하는 비효율 존재.

Spring Cache + Redis 어노테이션 기반 캐싱. 대상·TTL 분리 설계.

캐시 전략

캐시	TTL	대상 메소드	목적
`kisQuote`	5초	`KisMarketDataService.getQuote()`	동일 종목 연속 조회 (스캐너·포트폴리오 리프레시)
`kisBalance`	30초	`KisTradingService.getBalance()`	10개 전략이 동시 리밸런싱해도 잔고 API 1번만
`kisChart`	5분	`KisMarketDataService.getChartData()`	백테스트 반복 실행 시 100+ 호출 절감
`dartFundamentals`	24시간	FundamentalDataService (내부 Map 캐시)	재무제표는 분기/연 단위 → 하루 1회면 충분
`kospiUniverse`	1시간	KospiUniverseService	유니버스 리스트는 거의 불변

캐시 무효화 (정합성)

주문 성공 시 `@CacheEvict(value = "kisBalance")` 로 즉시 무효화 → 델타 계산 시 반드시 최신 잔고 참조. **stale 잔고로 매수/매도 계산 방지**.

효과 예시 (10개 전략 동시 실행 시)

	캐시 없음	Redis 캐시	절감
getBalance() 호출	10회	1회 (이후 캐시 히트)	-90%
getChartData() 호출	120회 (10 전략 × 평균 12자산)	30회 (중복 자산 제거)	-75%
리밸런싱 소요 시간	~45초	~12초	-73%

Redis 분산 락

`DistributedLock` 컴포넌트로 SET NX EX 패턴 구현. 수평 확장 시 동일 전략/포지션을 두 인스턴스가 동시 처리하는 race condition 방지. `@ConditionalOnBean(StringRedisTemplate.class)` 로 Redis 없을 때 로드 안 되게 가드.

Trade-off

- 캐시 TTL 길수록 API 부담 줄지만 stale 데이터 리스크 커짐 — 주문 flow 직전 반드시 evict.
- Redis 장애 시 fallback: `CACHE_TYPE=simple` 로 로컬 `ConcurrentMapCache` 전환.
- 주문 flow에는 캐시 적용 안 함 — 실제 호출 보장.

6. 이벤트 아키텍처 — Kafka

Problem — 주문·체결·틱 이벤트를 다양한 다운스트림(대시보드·분석·외부 시스템)이 필요로 하는데 기존엔 `EmailService.sendHtml()` 이나 직접 호출로 강하게 결합. 새 소비자 추가 시 비즈니스 로직 수정 필요. 비동기 처리 · 이력 보관 · 리플레이 불가.

Kafka 기반 이벤트 스트림 도입

- 3개 토픽: `trade-events` , `tick-events` , `audit-events`
- JsonSerializer + schemaVersion 필드로 버전 관리
- `EventPublisher` 는 fire-and-forget (Kafka 장애가 주문 흐름 막지 않음)
- `@KafkaListener` consumer는 `@ConditionalOnProperty` 로 토글 — 점진적 마이그레이션

토픽 설계

Topic	Producer	Consumer 후보	Key
`trade-events`	OcoOrderService (STOP_LOSS, TAKE_PROFIT) QuantExecutionService (REBALANCE)	분석 파이프라인 외부 리스크 시스템 규제 보고용 집계	symbol
`tick-events`	KisWebSocketClient (선택적 fan-out)	OcoOrderService (TickConsumer) 실시간 대시보드	symbol
`audit-events`	AuditLogService (선택적)	ELK/Splunk 적재 이상행동 탐지	user

TradeEvent 스키마

```
{
  "schemaVersion": 1,
  "eventType": "STOP_LOSS ¦ TAKE_PROFIT ¦ ORDER_PLACED ¦ STRATEGY_REBALANCE",
  "occurredAt": "2026-04-23T14:32:15",
  "source": "OcoOrderService",
  "strategyType": "VAA",
  "symbol": "069500",
  "side": "SELL",
  "quantity": 100,
  "price": 28500,
  "orderNo": "0000123456",
  "positionId": "uuid-...",
  "success": true,
  "message": "..."
}
```

왜 Kafka인가 (설계 의도)

- **Decoupling**: 주문 로직은 "이벤트 발행"만. 소비자 수 증가해도 주문 코드 불변
- **Replay**: 토픽 retention 7일로 장애 복구·분석 재실행 가능
- **Throughput**: 틱 이벤트가 장중 대량 발생해도 백프레셔 감당 (WS 직결보다 안정적)
- **Multi-consumer fan-out**: 같은 이벤트를 여러 consumer group이 독립적으로 소비

Trade-off

- 추가 인프라 (브로커 운영, 스토리지). 로컬 dev는 Kafka 없어도 EventPublisher가 no-op로 떨어져 영향 없음.
- Exactly-once 보장은 별도 구현 필요 (현재 at-least-once). Outbox 패턴으로 확장 가능.
- 순서 보장은 같은 key(symbol) 내 파티션 레벨만.

7. 컨테이너화 — Docker & docker-compose

7.1 Dockerfile — 멀티스테이지 빌드

Backend:

```
FROM eclipse-temurin:17-jdk-alpine AS build
COPY pom.xml → mvn dependency:go-offline  # 의존성 레이어 캐시
COPY src/ → mvn package -DskipTests

FROM eclipse-temurin:17-jre-alpine        # 최종 이미지 슬림
USER app                                   # 비-root
VOLUME /data                               # H2 파일 영속화
HEALTHCHECK → /actuator/health
ENTRYPOINT java $JAVA_OPTS -jar app.jar
```

Frontend:

```
FROM node:20-alpine AS build → npm run build
FROM nginx:1.27-alpine                     # SPA fallback + /api 프록시
COPY nginx.conf                            # React Router try_files
```

7.2 docker-compose

전체 스택 1 커맨드로 기동: `docker compose up -d`

서비스	이미지	포트	의존성
redis	redis:7-alpine	6379	—
kafka	bitnami/kafka:3.7 (KRaft)	9092	—
kafka-ui	provectuslabs/kafka-ui	9000	kafka
backend	빌드	8080	redis, kafka
frontend	빌드 (nginx)	80	backend

환경변수는 `.env` 파일로 주입 (KIS 키, DART 키, 알림 메일 등). `condition: service_healthy` 로 기동 순서 제어. Named volume으로 Redis/Kafka/H2 데이터 영속화.

8. 쿠버네티스 배포

8.1 리소스 구성

파일	리소스	핵심 결정
00-namespace	Namespace	논리적 격리
10-secrets	Secret + ConfigMap	민감 정보 분리 (SealedSecrets 권장)
20-redis	StatefulSet + headless Svc + PVC 2Gi	단일 인스턴스 (캐시 용도)
30-kafka	StatefulSet (KRaft) + PVC 10Gi	replicas=1 (POC) → 3 (prod)
40-backend	StatefulSet + PVC 5Gi	H2 파일 DB + KIS WS 단일 연결 → replicas=1
50-frontend	Deployment + HPA 2~5	stateless nginx → 수평 확장
60-ingress	Ingress + TLS	path 기반 라우팅 /api → backend, / → frontend

8.2 Probe 설계

```
backend:
  startupProbe:    /actuator/health       (30 × 10s = 최대 5분)  — 초기 DB 복구 시간 확보
  livenessProbe:   /actuator/health/liveness  (20s 주기)         — 데드락 감지
  readinessProbe:  /actuator/health/readiness (10s 주기)         — 트래픽 라우팅 제어
  resources:       512Mi~1.5Gi / 200m~1500m                     — KIS 피크 대비 여유
```

8.3 현 구성의 한계와 운영 로드맵

현재 제약

- Backend `replicas=1` — H2 파일 DB + KIS WS 단일 세션 제약
- Kafka `replicas=1` — 단일 장애점 (POC 단계)
- 다운타임 있는 배포 (rolling update 시 OCO 감시 일시 중단)

운영 확장 방안

1. **H2 → Postgres**: CloudNativePG 오퍼레이터. 수평 확장 가능.
2. **Leader election**: KIS WS 단일 세션은 Lease 리소스로 leader만 연결. 다른 pod는 standby.
3. **Kafka 3-broker**: replicas=3 + anti-affinity. 실제 운영 레벨.
4. **Service Mesh**: Istio + mTLS + Circuit Breaker.
5. **Observability**: Prometheus ServiceMonitor (Actuator metrics), Grafana 대시보드, Loki 로그.

9. 면접 예상 질의 & 답변 요지

Q1. "서버 재시작되면 OCO 포지션 어떻게 됩니까?"

기동 시 `@PostConstruct` 에서 H2 DB에 저장된 PENDING_*/ACTIVE 상태 포지션을 로드하고, KIS 잔고와 대조해 외부 매도로 증발된 케이스를 감지합니다. 그 후 각 포지션의 WS 재구독을 통해 감시를 재개합니다.
(실제 구현: `OcoOrderService.recoverFromDb()`)

Q2. "주문이 중복 발송될 가능성은?"

3중 방어입니다. (1) 전략 레벨 — lastRebalanceDate로 같은 날 중복 스케줄 스킵. (2) 주문 레벨 — KIS 미체결 조회로 symbol+side+quantity 매칭 주문 확인. (3) 네트워크 레벨 — 타임아웃 예외 발생 시 `reconcileUnknownOrder` 로 실제 접수 여부 검증.
추가로 Redis 분산 락이 준비되어 있어 수평 확장 시 동일 포지션 concurrent 처리를 막을 수 있습니다.

Q3. "WebSocket 끊기면 손절 못 하는 거 아닌가요?"

두 가지 방어선이 있습니다. (1) WS 클라이언트의 지수 백오프 재연결 + 기존 구독 자동 복원. (2) 60초 주기 폴백 폴링 — ACTIVE 포지션 종목의 현재가를 `getQuote()` 로 조회해 `onTick()` 을 재생시킵니다. WS가 정상 작동해도 중복 트리거는 `remainingQuantity=0` 체크로 방지됩니다.

Q4. "왜 Kafka/Redis를 넣었나요?"

Redis는 Rate limit 보호. KIS는 초당 18건 한도라 전략 10개가 동시 리밸런싱하면 쉽게 넘습니다. 잔고 조회를 30초 캐시하면 10번 칠 걸 1번만 칩니다.
Kafka는 주문·체결 이벤트를 다운스트림(분석, 외부 리스크 시스템, 대시보드)에 fan-out하기 위해서입니다. 강결합 메일 호출 대신 이벤트 버스를 통과시키면, 신규 consumer 추가 시 주문 로직을 수정 안 해도 됩니다. 장애 격리 효과도 있습니다 — Kafka가 죽어도 주문은 나갑니다 (fire-and-forget).

Q5. "테스트 커버리지는 얼마나 됩니까?"

전략 10종의 핵심 로직과 지표 계산(VAA 13612W 스코어, 모멘텀, SMA 등)을 38개 단위 테스트로 커버합니다. 팩터 전략은 Mockito로 FundamentalDataService를 스텁 처리하고 ROA+PER 랭킹·적자 기업 제외 등 경계 조건을 명시적으로 검증합니다. 외부 API 통합 테스트는 과금·rate limit 이슈로 제외하고 스테이징에서 수동 smoke test로 대체합니다.

Q6. "Backend 왜 단일 복제본이에요? 장애나면?"

두 가지 제약 때문입니다. (1) 현재 H2 파일 DB — Postgres 전환이 해결책. (2) KIS WebSocket은 계좌당 단일 세션만 지원.
운영 전환 시 Postgres로 옮기고, Backend는 leader election을 붙여 KIS WS는 leader 1개만 연결, 나머지는 standby로 두면 다중 복제본 가능합니다. 현 아키텍처는 POC 단계지만 K8s StatefulSet + PVC 로 설계해 놔서 인프라 변경은 최소입니다.

Q7. "배포 자동화·모니터링은?"

Docker 멀티스테이지 빌드 + docker-compose(로컬)/Kubernetes(운영). Actuator로 `/health`, `/prometheus` 노출해 Prometheus + Grafana 연동 준비. K8s probe로 자동 재시작·트래픽 제어. HPA로 frontend 자동 확장. CI/CD는 GitHub Actions 또는 ArgoCD로 GitOps 전환 가능한 상태입니다.

부록: 파일 구조

```
backtesting-tool/
├── backend/
│   ├── src/main/java/com/backtesting/
│   │   ├── controller/        — REST endpoints (5개)
│   │   ├── service/           — 비즈니스 서비스
│   │   │   ├── quant/         — 전략 10종 + 백테스트·실행 엔진
│   │   │   ├── kis/           — KIS API (HTTP · WebSocket)
│   │   │   ├── dart/          — DART · 재무 데이터
│   │   │   └── DistributedLock · DailyLossGuard · EmailService
│   │   ├── persistence/       — JPA 엔티티 + 리포지토리
│   │   ├── events/            — Kafka Event + Publisher + Consumer
│   │   ├── model/             — DTO
│   │   └── config/            — WebConfig · SecurityConfig · Cache · Health
│   ├── src/test/java/...      — JUnit 38 tests
│   ├── pom.xml
│   └── Dockerfile
├── frontend/
│   ├── src/components/        — React 컴포넌트
│   ├── Dockerfile             — nginx multi-stage
│   └── nginx.conf
├── k8s/
│   ├── 00-namespace.yaml
│   ├── 10-secrets.yaml        — Secret + ConfigMap
│   ├── 20-redis.yaml          — StatefulSet
│   ├── 30-kafka.yaml          — StatefulSet (KRaft)
│   ├── 40-backend.yaml        — StatefulSet + PVC
│   ├── 50-frontend.yaml       — Deployment + HPA
│   ├── 60-ingress.yaml
│   └── README.md
├── docker-compose.yml
├── .env.example
└── docs/
    └── (this document)
```